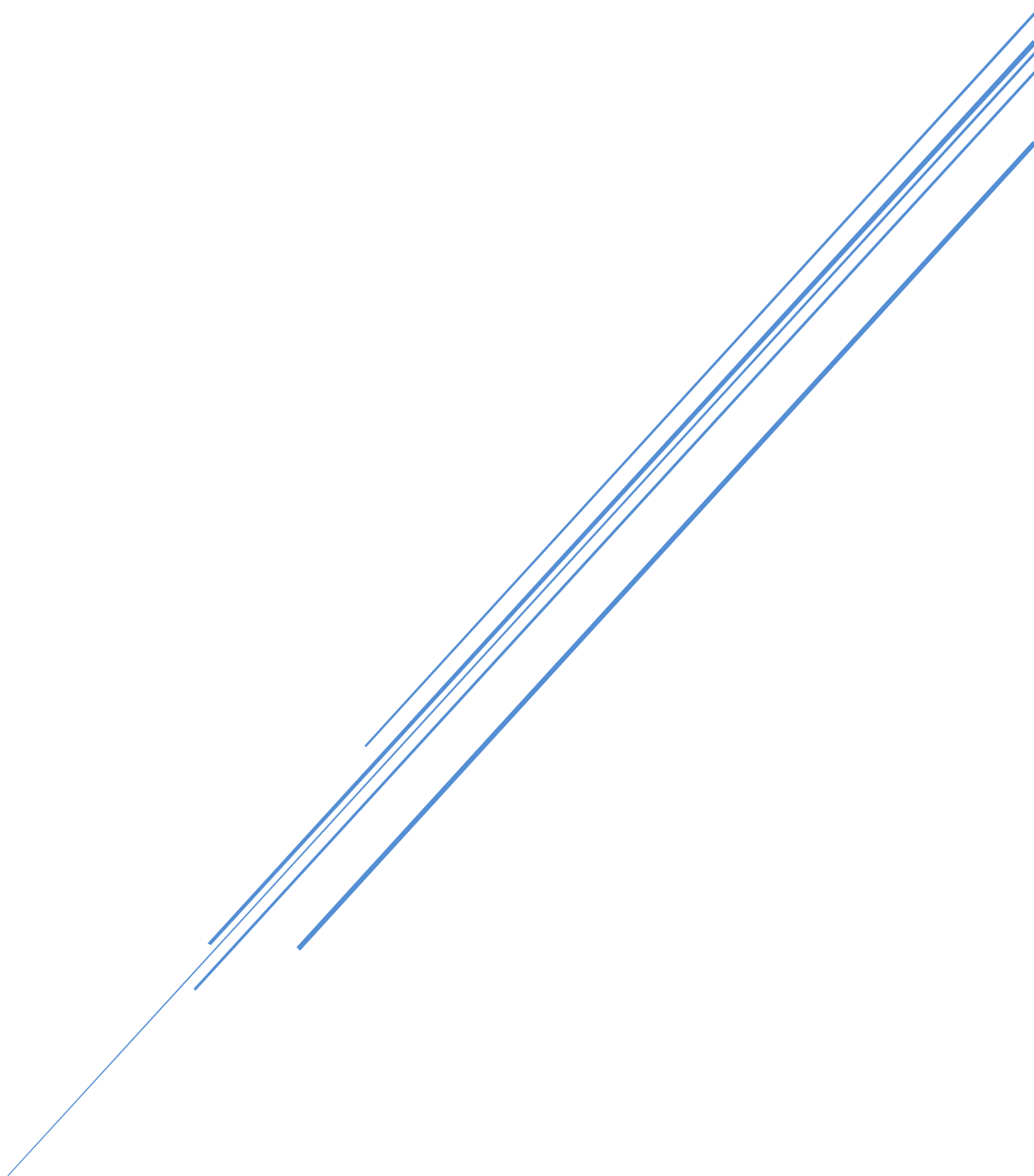


VAPT REPORT

target1.local

demo-web-app

lab.example



REDTEAM CYBERSECURITY LABS
Midhilesh C

TABLE OF CONTENTS

1 EXECUTIVE SUMMARY	2
1. Overview	2
2. Summary of Findings	2
3. Risk Score	2
2.SCOPE OF TESTING	3
3.METHODOLOGY	4
1. Testing phases	4
2. Tools Used	5
4.DETAILED VULNERABILITY ANALYSIS	6
4.1 Improper Directory Access & Exposure of Backend Files	6
4.2 Missing Security Headers & Insecure Cookie Configuration	9
4.3 Cross-Site Scripting (Reflected & Stored XSS)	11
5.REMEDIATION PLAN	16
6.CONCLUSION AND RECOMMENDATION	17
7.APPENDICES	18
1. Test Case Results	18
2. Tools and versions Used	18

1. EXECUTIVE SUMMARY

1. Overview

The Vulnerability Assessment and Penetration Testing (VAPT) was conducted to evaluate the security posture of the target web application. The primary objective of this assessment was to identify security vulnerabilities, misconfigurations, and potential attack vectors that could compromise the confidentiality, integrity, and availability of the application and its associated data.

The assessment was performed using a black-box testing approach, simulating real-world attacker behavior without prior access or credentials. A structured methodology combining automated scanning and manual verification was followed. Automated testing was carried out using tools such as Dirsearch and Burp Suite, while manual analysis was performed to validate findings, assess real-world impact, and eliminate false positives.

2. Summary of Findings

Multiple security vulnerabilities were identified during the assessment and were categorized based on their severity into **High, Medium, and Low risk levels**.

- A **High-risk Cross-Site Scripting (XSS) vulnerability**, including both reflected and stored XSS, was identified. This issue allows attackers to inject and execute malicious scripts in users' browsers, potentially leading to session hijacking and account compromise.

- A **Medium-risk security misconfiguration** was observed due to missing HTTP security headers and insecure cookie attributes. This increases the likelihood of session hijacking, clickjacking, and man-in-the-middle attacks.

- A **Low-risk information disclosure issue** was detected where internal directories and backend files were publicly accessible, exposing application structure and sensitive resources that could assist attackers during reconnaissance.

Based on the severity and exploitability of the identified issues, the overall security posture of the application is assessed as **Medium Risk**, requiring prompt remediation of high-impact vulnerabilities to prevent potential exploitation.

3. Risk Score

The identified vulnerabilities were evaluated using the **OWASP Risk Rating Methodology**, considering both the **likelihood of exploitation** and the **potential impact** on the application.

OWASP Risk Formula:

Risk = Likelihood × Impact

Vulnerability	Likelihood	Impact	Risk Score	Risk Level
Cross-Site Scripting (Reflected & Stored)	3	3	9	High
Missing Security Headers & Insecure Cookies	2	3	6	Medium
Improper Directory & Backend Exposure	1	3	3	Low

2. SCOPE OF TESTING

Target

Primary Domains:

target1.local

demo-web-app

lab.example

Subdomains Discovered:

None explicitly identified during the assessment.

Testing Methodology

The Vulnerability Assessment and Penetration Testing (VAPT) was conducted using a **Black Box Testing approach**, where no prior credentials, source code, or internal system details were provided. This approach simulates real-world attacker behavior to evaluate the external security posture of the target applications.

The assessment was performed in alignment with the **OWASP Top 10 security risks** and established industry best practices. A combination of **manual testing techniques and limited automated analysis** was used to identify security vulnerabilities and misconfigurations.

The following activities were performed during testing:

Directory enumeration to identify exposed directories such as /uploaded/ and /bin/

Manual inspection of publicly accessible backend files

Authentication and authorization testing on admin login portals

Input validation testing to identify Cross-Site Scripting (XSS) vulnerabilities

Verification of improper access controls and information disclosure

All findings were manually verified to confirm exploitability and reduce false positives.

Exclusions

The following items were excluded from the scope of this assessment:

Third-party services, integrations, or platforms not owned or directly controlled by the target organizations

External payment gateways

Social media authentication providers

Email service providers

Content Delivery Networks (CDNs)

Denial-of-Service (DoS) or stress testing

Source code review and internal network testing

Any vulnerabilities related to excluded components were not tested further and were not considered part of this assessment.

3. METHODOLOGY

1. Testing Phases

The security assessment followed a structured methodology covering critical security areas based on OWASP Top 10 and industry best practices. The following testing phases were conducted:

No.	Testing Phases	Description
1.	Information Gathering	Identified publicly available information about the target, including subdomains, technologies, and open ports.
2.	Configuration & Deployment Management Testing	Checked for security misconfigurations in server settings and third-party services.
3.	Identity & Access Management Testing	Assessed how user identities are managed and validated.
4.	Authentication & Authorization Testing	Tested login mechanisms, privilege escalation risks, and access control enforcement.
5.	Session Management Testing	Evaluated session handling, cookies, and session expiration policies.
6.	Input Validation Testing	Checked for SQL Injection (SQLi), Cross-Site Scripting (XSS), and other input-related flaws.
7.	Error Handling Testing	Verified improper error messages that reveal sensitive system details.
8.	Weak Cryptography Testing	Analysed encryption methods and data transmission security.
9.	Business Logic Testing	Identified flaws in workflow that attackers could manipulate.
10.	Client-Side Testing	Tested JavaScript execution, DOM-based attacks, and security headers.
11.	API Testing	Examined API endpoints for authentication, authorization, and data exposure risks.

3.2 Tools Used

The following security tools were used to identify vulnerabilities, analyze network configurations, and conduct penetration testing during the assessment:

Tool Used	Description
Kali Linux	A Linux distribution containing various security tools for ethical hacking and vulnerability assessment.
VirtualBox / VMware	Virtualization platforms used to run Kali Linux in an isolated environment.
Burp suite	A powerful web vulnerability scanner and proxy tool used for intercepting, analysing, and modifying HTTP requests.
Subfinder	A subdomain enumeration tool used to find subdomains of a target website.
Nmap	Network scanning tool to discover open ports and running services.
Nikto	A web server scanner that detects vulnerabilities such as outdated software, misconfigurations, and potential security issues in web applications.
Wappalyzer	A browser extension and command-line tool that identifies web technologies, including CMS, frameworks, and analytics.

4.DETAILED VULNERABILITY ANALYSIS

4.1 Improper Directory Access & Exposure of Backend Files

Severity: Low

CVSS v3.1 Score: 3.5

OWASP Category: A05:2021 – Security Misconfiguration

Proof of Concept – Steps to Reproduce

- Conduct directory enumeration using **dirsearch**:

```
dirsearch -u https://target1.local.in
```

- Observe discovered directories such as /uploaded/ and /bin/ in the scan results.

- Open a web browser and manually access the discovered directories:

```
https://target1.local.in/uploaded/ https://target1.local.in/bin/
```

- Verify that directory contents and backend files are displayed without authentication.

Impact

The application allows **public access to internal directories** such as /uploaded/ and /bin/, resulting in information disclosure. These directories expose files, documents, and server-side scripts that should not be accessible to unauthenticated users.

Exposure of backend scripts and uploaded files enables attackers to understand internal application structure and backend processing logic, which can be leveraged for further attacks.

Affected Components

- /uploaded/ directory
- /bin/ directory

Web server directory indexing configuration

PROOF OF CONCEPT

Index of /uploaded/

Filter Name

Name	Last Modified	Size
Parent Directory		
1593073366.jpg	2020-06-25 11:52	249k
1601895889.pdf	2020-10-05 14:34	1106k
1603102781.pdf	2020-10-19 13:49	644k
1611990881.pdf	2021-01-30 11:44	1721k
1611991121.pdf	2021-01-30 11:48	481k
1612856781.pdf	2021-02-09 12:16	202k
1613376161.pdf	2021-02-15 12:32	289k
1614233362.pdf	2021-02-25 10:39	247k
1618472120.pdf	2021-04-15 11:05	80k
1624949567.pdf	2021-06-29 10:22	452k
1626430646.pdf	2021-07-16 13:47	120k
1627542750.pdf	2021-07-29 10:42	204k
1627546968.pdf	2021-07-29 11:52	230k
1628753846.jpg	2021-08-12 11:07	326k
1629200927.pdf	2021-08-17 15:18	351k
1629267087.pdf	2021-08-18 09:41	323k

Index of /bin/

Name	Last Modified	Size
Parent Directory		
MailHandler.ashx	2014-09-21 00:08	2k
MailHandler.php	2019-09-23 14:07	2k

Proudly Served by LiteSpeed Web Server at InCollege.ac.in Port 443

Remediation

- Disable directory listing in server configuration
- Restrict public access to internal directories
- Move sensitive files outside the web root
- Implement strict access control rules

4.2 Missing Security Headers & Insecure Cookie Configuration

Severity: Medium

CVSS v3.1 Score: 6.0

OWASP Category: A05:2021 – Security Misconfiguration

Impact

The application lacks essential HTTP security headers and uses **insecure cookie attributes**, weakening browser-based protections. Missing headers reduce defense against XSS and clickjacking, while insecure cookies increase the risk of session hijacking.

Affected Components

HTTP response headers

Session cookies

Proof of Concept – Steps to Reproduce

- Intercept application traffic using **Burp Suite**.
- Send a request to the server and inspect the **HTTP response headers**.
- Identify missing security headers such as Content-Security-Policy, X-Frame-Options, and X-Content-Type-Options.
- Observe the Set-Cookie header lacking Secure and HttpOnly flags.

Potential Exploitation

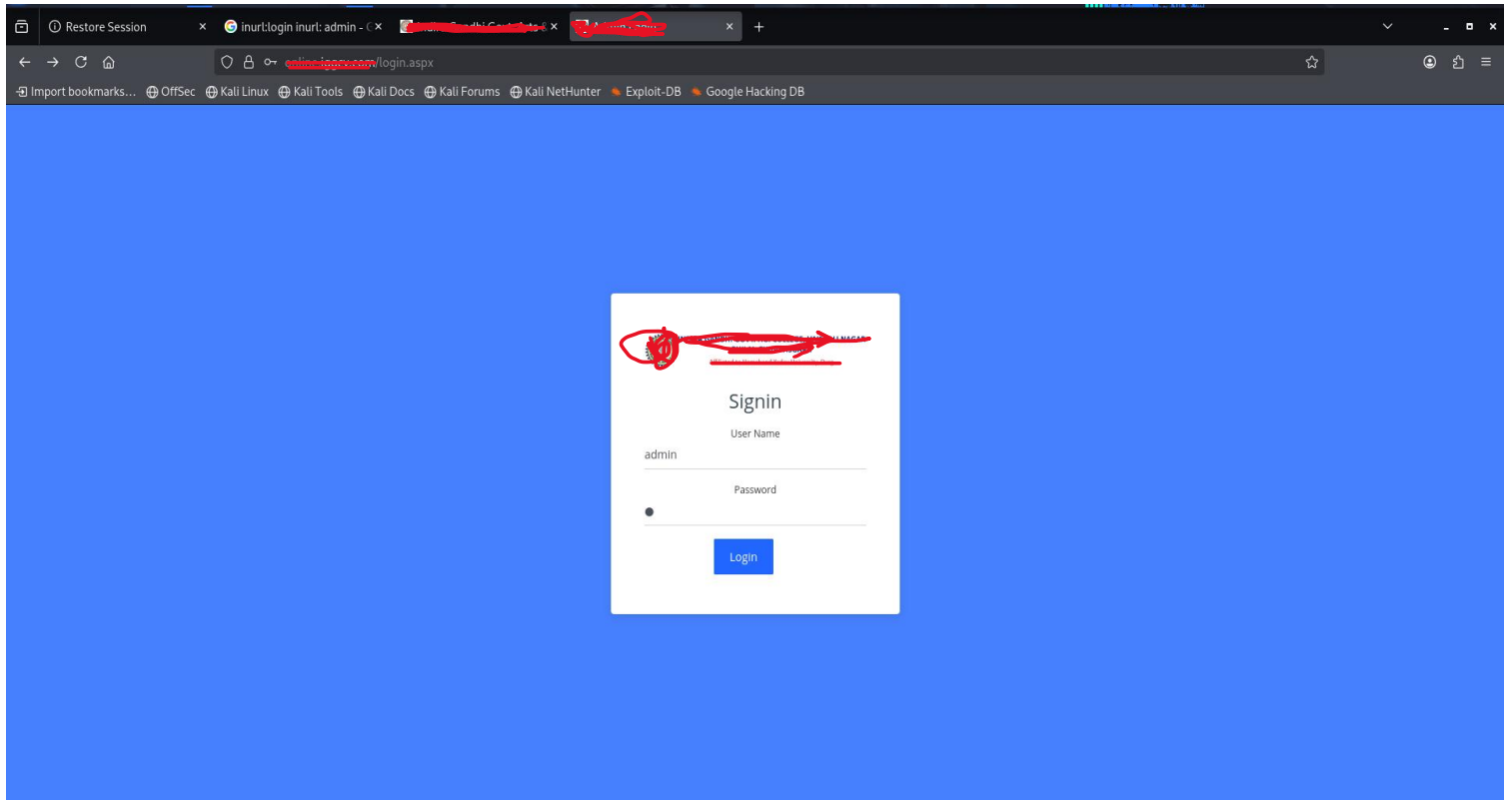
Session hijacking

Clickjacking attacks

Increased XSS impact

Man-in-the-middle attacks

Proof of Concept



ONLINE ADMISSION

Govt. College Varshali Nagar

Dashboard Admision Settings Master Master

Select Year
2025

Class Wise Admission Form

SN	COURSE_NAME	PART	REGISTRATIONS
1	B.A.	2	241
2	B.A.	3	140
3	B.Com	2	359
4	B.Com	3	347
5	B.Com Computer Application	3	18
6	B.Sc. (Chem. + Botany + Zoology)	2	52
7	B.Sc. (Chem. + Botany + Zoology)	3	52
8	B.Sc. (Indust. Micro Bio + Chem. + Botany)	2	22
9	B.Sc. (Indust. Micro Bio + Chem. + Botany)	3	14
10	B.Sc. (Math + Physics + Chem.)	2	9
11	B.Sc. (Math + Physics + Chem.)	3	9
12	B.Sc. (Math + Physics + Computer Science)	2	30
13	B.Sc. (Math + Physics + Computer Science)	3	35
14	B.Sc. Home Science	2	6
15	B.Sc. Home Science	3	6
16	M.A. Economics	3	17
17	M.A. English Literature	3	14
18	M.A. Political Science	3	15
19	M.Com.	3	17
20	M.Sc. Chemistry	3	9
	Total		1412

Class Wise Admitted

SN	COURSE_NAME	PART	REGISTRATIONS
1	B.A.	3	130
2	B.Com	3	112
3	B.Sc. (Chem. + Botany + Zoology)	3	47
4	B.Sc. (Indust. Micro Bio + Chem. + Botany)	3	13
5	B.Sc. (Math + Physics + Chem.)	3	9
6	B.Sc. (Math + Physics + Computer Science)	3	34
7	B.Sc. Home Science	3	5
	Total		350

Remediation

- Implement missing HTTP security headers
- Set Secure, HttpOnly, and SameSite flags for cookies
- Enforce HTTPS across the application

4.3 Cross-Site Scripting (Reflected & Stored XSS)

Severity: High

CVSS v3.1 Score: 8.0

OWASP Category: A03:2021 – Injection

Impact

The application fails to properly validate and encode user input, allowing both reflected and stored XSS attacks. This enables attackers to execute malicious JavaScript in victims' browsers, potentially leading to account compromise.

Affected Components

Input fields

Search parameters

Stored content rendering

Proof of Concept – Steps to Reproduce

- Identify an input field (search, comment, or form submission).
- Inject a JavaScript payload into the input field:

```
<script>alert(1)</script>
```
- Submit the input and observe script execution (reflected or stored).
- Reload or revisit the affected page to confirm persistent execution (stored XSS).

Potential Exploitation

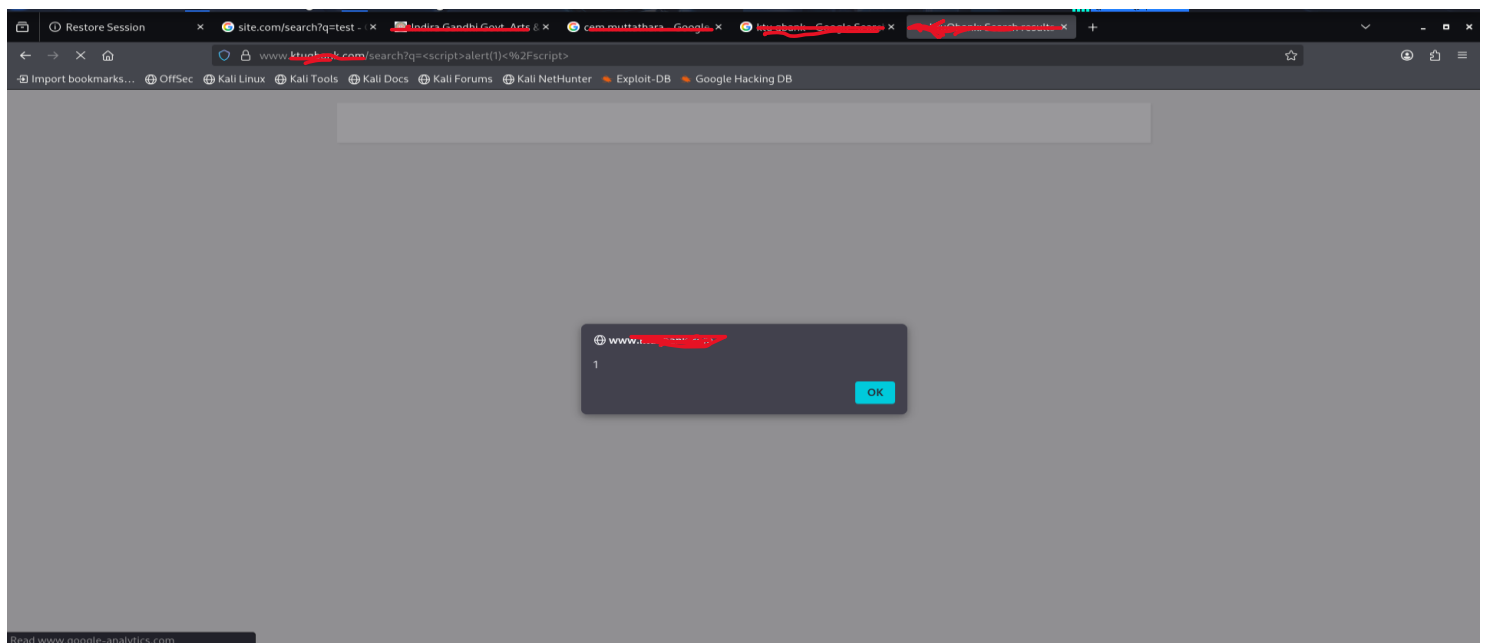
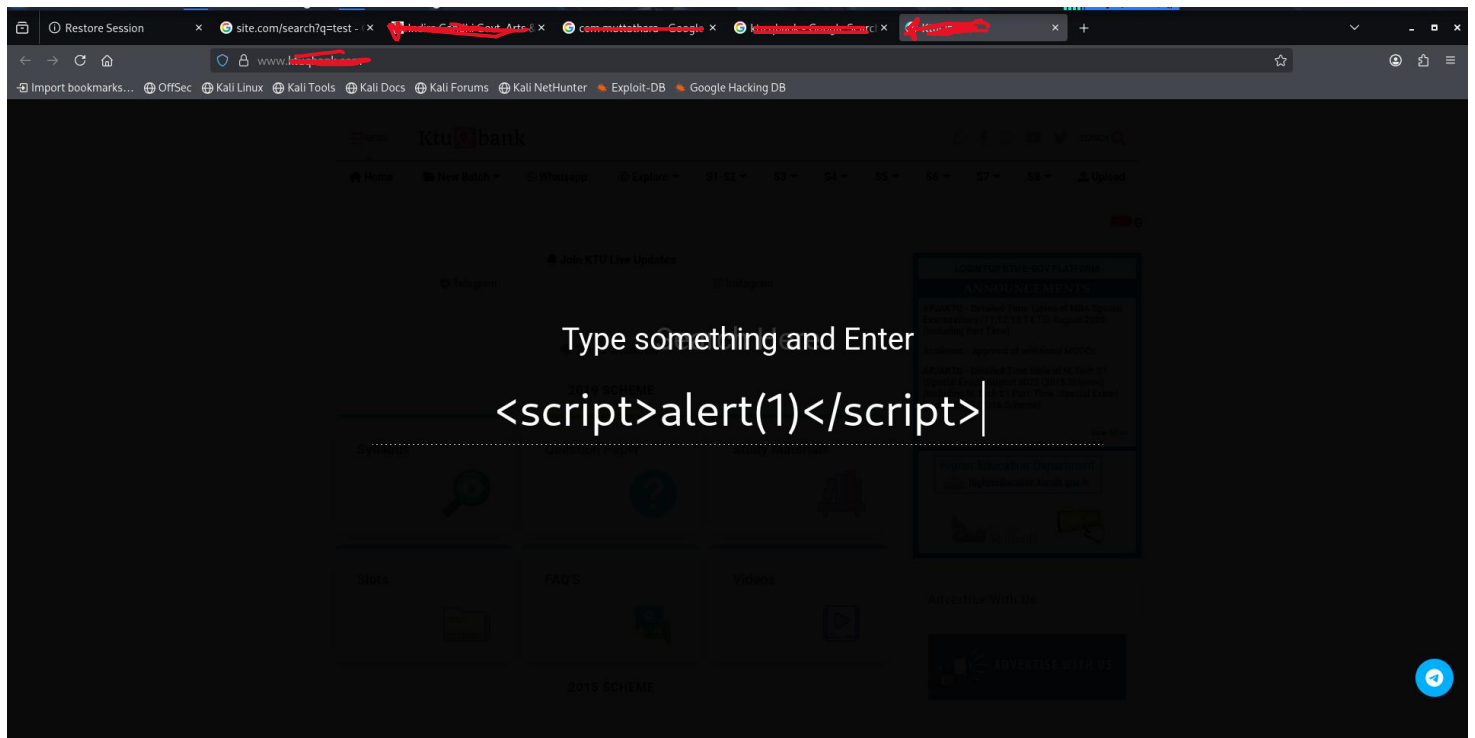
Session hijacking

Credential theft

Admin account compromise

Phishing attacks

Screenshot



Remediation

- Validate and sanitize all user inputs
- Encode output before rendering
- Implement Content Security Policy (CSP)
- Use secure development frameworks

5. Remediation Plan

Priority 1 – High (Immediate Action Required)

Fix Cross-Site Scripting (XSS) Vulnerabilities

Implement strict input validation and output encoding for all user inputs.

Sanitize and encode data before rendering it in web pages.

Deploy a strong Content Security Policy (CSP) to restrict script execution.

Re-test all input fields after applying fixes.

Priority 2 – Medium (Short-Term Action)

Secure Cookies and HTTP Security Headers

Set Secure, HttpOnly, and SameSite attributes for all session cookies.

Implement missing HTTP security headers such as CSP, X-Frame-Options, HSTS, and X-Content-Type-Options.

Enforce HTTPS across the entire application to prevent man-in-the-middle attacks.

Priority 3 – Low (Preventive Action)

Restrict Directory Access and Information Disclosure

Disable directory listing for internal directories such as /uploaded/ and /bin/.

Restrict public access to backend files and sensitive directories.

Perform periodic audits to identify and remove exposed files.

6. Conclusion & Recommendations

- The Vulnerability Assessment and Penetration Testing (VAPT) identified multiple security weaknesses across the target applications, indicating gaps in secure configuration, input validation, and access control mechanisms.
- A high-risk Cross-Site Scripting (XSS) vulnerability was discovered, which could allow attackers to execute malicious scripts and compromise user sessions and sensitive data.
- Medium-risk issues such as missing HTTP security headers and insecure cookie attributes increase the likelihood of session hijacking, clickjacking, and man-in-the-middle attacks.
- Low-risk vulnerabilities related to exposed directories and backend files were observed, which could assist attackers during reconnaissance and attack planning.
- Immediate remediation of high-priority vulnerabilities is strongly recommended to reduce the overall attack surface and prevent exploitation.
- Implementation of proper input validation, output encoding, and a strong Content Security Policy (CSP) is essential to mitigate client-side attacks.
- Security headers and cookie configurations should be hardened by enabling Secure, HttpOnly, and SameSite attributes and enforcing HTTPS across the application.
- Directory listing should be disabled, and access to internal files and backend resources should be restricted using proper permissions and access controls.
- Regular security assessments, vulnerability scanning, and manual testing should be integrated into the software development lifecycle to identify new risks early.
- Adopting secure coding practices, continuous monitoring, and an incident response strategy will significantly improve the long-term security posture of the application.

7. Appendices

1. Test Case Results

Test Case	Methodology	Expected Result	Actual Result	Status
Weak Admin Credentials	Attempted login with default credentials	Access denied	Admin panel accessible with default credentials	Vulnerable
Exposed Admin Page	Accessed /admin	Restricted access	Admin page accessible without authentication	Vulnerable
Insecure Cookie Handling	Scanned with Nikto for missing security flags	Cookies set with Secure and HttpOnly flags	Cookies lack Secure and HttpOnly flags	Vulnerable
Directory Listing Enabled	Performed directory enumeration using Dirsearch	Directory access should be restricted	Sensitive directory accessible publicly	Vulnerable

7.2 Tools and Versions Used

- Kali Linux – 2024.1
- VMware – 17.5.0
- Burp Suite Pro – 2025.1
- DIRB – 2.22
- Nikto – 2.5.0
- Nmap – 7.93
- Wappalyzer CLI – 6.11.3